PyMalGraph: Knowledge-Graph-Augmented Static Analysis for Malicious Python Package Detection

Bui Viet Dung

Faculty of Information Assurance, FPT University, Hanoi, Vietnam

dungbvhe186274@fpt.edu.vn

Abstract

The proliferation of software supply chain attacks via the Python Package Index (PyPI) demands scalable, semantically precise detection mechanisms that can operate purely on source distributions without execution. We present PyMalGraph, a novel pipeline that integrates (i) a curated suite of 47 CodeQL semantic queries targeting six malware behavioural dimensions, (ii) a formally defined Observation *schema mapped to MITRE ATT&CK techniques, (iii) a typed heterogeneous knowledge graph $\mathcal{G}_p = (\mathcal{V}_p, \mathcal{E}_p, \phi, \psi)$ per package, and (iv) a graph topology embedding $\Phi\colon \mathcal{G}_p \to \mathbb{R}^{16}$ feeding a supervised binary classifier.*

*We evaluate on 532 real-world packages (234 malicious, 298 benign). The training protocol uses a family-aware split to reduce version-level leakage, while primary operational metrics are reported on all successfully scanned samples ($n = 524$). The Random Forest classifier achieves **93.3%** accuracy, **96.7%** precision, **88.0%** recall, and **92.2%** F1-score on the all-sample scope. For reference, the family-aware holdout test yields 84.7% accuracy, 88.1% precision, 75.4% recall, and 81.3% F1. All four predefined operational KPIs are satisfied. Feature importance analysis demonstrates that graph-structural signals (density and technique-diversity features) contribute 26.7% of total discriminative power, validating the knowledge-graph representation beyond raw CodeQL counts. We provide a full analysis of 28 false negatives and 7 false positives, identify the dominant attack technique T-DYN-EVAL (30,928 occurrences), and release the complete pipeline as a reproducible artifact.*

Index Terms—*software supply chain security, malware detection, CodeQL, knowledge graph, static analysis, graph embedding, PyPI, ATT&CK.*

I. INTRODUCTION

The Python Package Index (PyPI) serves over 500,000 packages and processes billions of downloads annually, making it a high-value target for adversarial supply-chain injection. Unlike traditional malware distributed via executables, *package-level* attacks exploit the trusted distribution infrastructure itself: a single malicious package transitively poisons every downstream project. Guo et al. [1] conducted a large-scale empirical study of 4,669 malicious PyPI packages and found that over 74% target user projects at install time, with command execution and information stealing as the two dominant attack classes. Shariffdeen et al. [2] further document that state-of-the-art packages chain encoding, encryption, and dynamic execution to conceal payloads, motivating the need for semantic-level analysis beyond pattern matching.

Detection is non-trivial for three structural reasons. *Execution timing:* malicious code typically runs in `setup.py` or `__init__.py` at install time, not during ordinary import, so dynamic sandboxes observe an artefact that behaves benignly post-install. *Obfuscation depth:* state-of-the-art samples chain Base64 decoding, AES decryption, and `eval(exec(...))` to conceal payloads from string-matching scanners—a pattern documented across over 53% of malicious packages by Guo et al. [1]. *Volume:* PyPI receives thousands of new packages daily; manual review is intractable.

Existing tools occupy two extremes. *Signature-based* scanners (Bandit [3], pattern rules) achieve low false-positive rates on known samples but fail on novel obfuscation. *Metadata-based* classifiers [4] are fast but miss code-level deception. Recent LLM-based approaches [5–7] achieve high accuracy but are computationally expensive and offer limited interpretability.

We bridge this gap with PyMalGraph, a *semantically-aware static analysis* system that reasons at the level of program structure rather than text. The pipeline proceeds through four stages: (1) CodeQL analysis producing structured findings from AST-level taint and dataflow reasoning; (2) lifting findings into a typed *knowledge graph* that encodes co-occurrence of indicators and ATT&CK techniques; (3) topology embedding to a fixed-dimension feature vector; (4) binary classification with interpretable feature importances.

Our design is directly informed by two prior works. Ryan et al. [8] provide a statement-level taxonomy of 47 malicious indicators across 7 behavioural groups for PyPI packages, establishing the semantic vocabulary that our CodeQL queries operationalise. Shariffdeen et al. [2] (HERCULE) demonstrate that CodeQL inter-package analysis can detect PyPI malware with precision 0.966 and recall 0.933; our work extends this by lifting CodeQL findings into a knowledge graph and showing that graph topology adds discriminative power beyond raw observation counts.

Contributions.

1. A formally specified observation model and knowledge-graph schema with category-theoretic typing (Section 3).
2. A curated suite of 47 CodeQL queries with 100% compile coverage, covering six ATT&CK-aligned attack dimensions, grounded in the indicator taxonomy of Ryan et al. [8] (Section 4).
3. A graph topology embedding Φ and a proof that it is invariant to node renaming (Proposition 2) (Section 3.3).
4. Empirical results on 532 packages with detailed error analysis, feature attribution, and comparison to related systems (Section 6).
5. Discussion of the theoretical limits of static analysis for evasion resistance, with open problems (Section 8).

II. BACKGROUND AND RELATED WORK

A. Empirical Studies of PyPI Malware

Guo et al. [1] performed the first large-scale empirical study specifically targeting the PyPI malware ecosystem, analysing 4,669 confirmed malicious packages collected between 2017–2023. Their taxonomy identifies five primary behavioural classes (command execution, information stealing, file operations, code execution, remote control) and documents that malicious code is predominantly embedded in `setup.py` (86% of indicators) and activated at install time—an empirical finding that directly motivates our query focus on install-time hooks. They further show that over 50% of packages exhibit multi-behaviour characteristics, validating the value of a multi-dimensional observation schema.

Ryan et al. [8] provide a *statement-level* taxonomy of malicious indicators in PyPI, manually annotating 370 packages at the source line level and deriving 47 distinct indicator types across 7 groups (Defense Evasion, Metadata Anomaly, System Access, Shell Execution, Module Execution, and Network), with inter-annotator agreement $\kappa = 0.81$. This taxonomy is the direct precursor to our indicator vocabulary $\mathcal{I}$ and the six attack dimensions covered by our CodeQL query suite (Table 1). Their finding that sequential attack patterns recur predictably motivates our knowledge-graph representation of indicator co-occurrence.

Zhou et al. [9] construct MALGRAPH, a knowledge graph linking over 9,000 malicious packages from six industry and academic sources through four relationship types (name similarity, author overlap, C2 overlap, code overlap). Their finding that only 39.27% of packages appear in any single database and that 28 repeated malicious packages serve as camouflage for 1,354 others highlights the importance of relational analysis—a perspective our per-package knowledge graph captures at the indicator and technique level.

B. Static Analysis Approaches for PyPI

Shariffdeen et al. [2] (HERCULE) is the closest prior system to PyMalGraph. HERCULE applies 22 CodeQL queries in an inter-package analysis pipeline that additionally performs AST-level integrity checking between published artifacts and their source repositories. It achieves precision 0.966, recall 0.933, and F1 0.949 on 3,480 packages— the strongest published results from a static-CodeQL-based detector at the time of writing. PyMalGraph extends this line of work by (i) increasing the query suite to 47 queries aligned with Ryan et al.'s taxonomy [8], (ii) lifting CodeQL findings into a typed knowledge graph, and (iii) demonstrating that graph topology features contribute 26.7% of discriminative power beyond raw observation counts.

Schreiber et al. [10] conduct a large-scale application of CodeQL's `security-and-quality` suite to 7,703 Python files, identifying 4,241 CWE instances and demonstrating that CodeQL's interprocedural dataflow analysis can surface security-relevant patterns at repository scale. Their methodology validates the use of CodeQL as an industrial-strength foundation for code-level security analysis.

Rule-based tools including Bandit [3], Semgrep [11], and GuardDog [12] provide intra-procedural AST pattern matching and metadata heuristics respectively. Zhang et al. [13] (Cerebro) improve on these by building cross-ecosystem (NPM + PyPI) behavior-sequence models, achieving roughly 10% precision and 7–8% recall gains over prior SOTA; however, Cerebro's approach requires a language model fine-tuned on behavioral sequences, whereas PyMalGraph is interpretable and requires only a Random Forest operating on a 16-dimensional graph embedding.

C. Knowledge Graphs in Malware Analysis

Knowledge graphs have been applied to malware analysis at multiple granularities. Rastogi et al. [14] propose MALOnt, an ontology with 68 classes and 31 properties for structuring malware threat intelligence, achieving 91.16% accuracy in automated threat analysis. Jing et al. [15] construct APT malware knowledge graphs from 9,019 samples spanning 10 threat-actor groups, using multi-stage graph clustering and meta-path guided embedding to achieve attribution accuracy of 91.16% and AUC 98.99%. Both works construct graphs from *reports or binary features*; PyMalGraph instead constructs graphs directly from *static code analysis findings*, providing evidence-linked technique attribution at the per-package level with source-line provenance.

MITRE ATT&CK [16] provides the adversarial technique taxonomy (196 techniques, 14 tactics) that our observation schema maps to, enabling the knowledge graph to be interpreted within an established threat intelligence framework.

D. LLM-Based and Hybrid Detection

Several concurrent systems apply large language models to PyPI malware detection. Zeshan et al. [5] (LAMPS) use a four-agent pipeline combining LLaMA-3 and fine-tuned CodeBERT, reporting F1 97.77% on balanced datasets. Zhang et al. [6] (PyGuard) use LLM-generated semantic summaries and hierarchical behavior mining to achieve accuracy 99.50%. Guo et al. [7] (IntelGuard) augment LLM reasoning with RAG over 8,000+ threat intelligence reports, reaching accuracy 99.0%. Samaana et al. [4] present a metadata- and code-feature ML classifier on 5,331 packages,

reporting strong signals from URL anomalies and license mismatches. Guo et al. [17] (IntelliRadar) focus on intelligence aggregation rather than detection, building a 20,692-package database from 21 public sources that could serve as a future training corpus.

These LLM-based systems achieve higher raw accuracy on their respective test sets but at considerably higher computational cost and with reduced interpretability relative to PyMalGraph. More importantly, they operate on balanced benchmark splits whereas PyMalGraph explicitly evaluates under a family-aware protocol to prevent version leakage.

III. FORMAL SPECIFICATION

We develop the formal foundations of PyMalGraph, establishing definitions used throughout the pipeline.

A. Observation Schema

Definition 1 (Observation)**.** An *observation* is a tuple $o = \langle id, q, \iota, \tau, s, f, e\rangle$ where:

- $id \in \mathbb{N}$ is a unique identifier.
- $q \in \mathcal{Q}$ is the CodeQL query that produced it.
- $\iota \in \mathcal{I}$ is the *indicator code* (e.g. `DEF-005`), grounded in the indicator vocabulary of Ryan et al. [8].
- $\tau \in \mathcal{T}$ is the ATT&CK-aligned *technique code* (e.g. `T-OBF-EMBED`).
- $s \in \{critical, high, medium, low\}$ is the severity.
- $f \in \mathcal{F}$ is the source file path.
- $e \in \mathcal{E}_{\text{loc}}$ is the evidence location (file, line range, code snippet).

Let $\mathcal{O}(p)$ denote the multiset of all observations for package p. We write $|\mathcal{O}(p)|$ for the total observation count and $\mathcal{O}_\tau(p) = \{o \in \mathcal{O}(p) \mid o.\tau = \tau\}$ for technique-specific subsets.

Definition 2 (Indicator vocabulary & Technique vocabulary)**.** $\mathcal{I}$ is a finite set of indicator codes partitioned as $\mathcal{I} = \mathcal{I}_{\text{DEF}} \cup \mathcal{I}_{\text{MET}} \cup \mathcal{I}_{\text{SYS}} \cup \mathcal{I}_{\text{EXS}} \cup \mathcal{I}_{\text{EXM}} \cup \mathcal{I}_{\text{NET}}$ corresponding to defense evasion, metadata, system, execution shell, execution module, and network categories respectively—reflecting the six behavioural groups identified by Ryan et al. [8]. The mapping $\mu\colon \mathcal{I} \to \mathcal{T}$ assigns each indicator to exactly one ATT&CK technique; multiple indicators may share a technique.

B. Knowledge Graph Definition

Definition 3 (Package Knowledge Graph)**.** The knowledge graph for package p is a typed directed multigraph $\mathcal{G}_p = (\mathcal{V}_p, \mathcal{E}_p, \phi_p, \psi_p, \lambda_p)$ where:

- $\mathcal{V}_p$ is the node set with type function $\phi_p\colon \mathcal{V}_p \to \mathfrak{N}$, $\mathfrak{N} = \{\text{pkg, obs, ind, tech, file, evid}\}$.
- $\mathcal{E}_p \subseteq \mathcal{V}_p \times \mathcal{V}_p$ is the edge set with type $\psi_p\colon \mathcal{E}_p \to \mathfrak{R}$, $\mathfrak{R} = \{\texttt{has_obs}, \texttt{has_ind}, \texttt{has_tech}, \texttt{in_file}, \texttt{has_evid}\}$.
- $\lambda_p\colon \mathcal{V}_p \to \bigcup_k \mathcal{A}_k$ assigns attribute dictionaries to nodes (severity, code, name, …).

The graph is constructed as follows. Let $p^* \in \mathcal{V}_p$ be the unique package node. For each observation $o_i \in \mathcal{O}(p)$, create node v_i with $\phi_p(v_i) = \text{obs}$ and add edge (p^*, v_i) of type `has_obs`. For the indicator $\iota = o_i.\iota$, obtain or create v_ι with $\phi_p(v_\iota) = \text{ind}$ and add $(v_i, v_\iota) : \texttt{has_ind}$. Similarly for technique, file, and evidence nodes. The construction algorithm runs in $O(|\mathcal{O}(p)| \cdot \alpha(|\mathcal{V}_p|))$ time (nearly linear via union-find for node deduplication).

Proposition 1 (Bounded graph size)**.** For a package with $n = |\mathcal{O}(p)|$ observations, $|\mathcal{V}_p| \le n + |\mathcal{I}| + |\mathcal{T}| + |\mathcal{F}_p| + n + 1$ and $|\mathcal{E}_p| \le 5n$, where $\mathcal{F}_p$ is the set of distinct files in p. Hence both grow *linearly* in n.

C. Graph Topology Embedding

Definition 4 (Topology embedding)**.** The embedding $\Phi\colon \mathcal{G}_p \to \mathbb{R}^{16}$ maps a package knowledge graph to a real-valued feature vector. Formally,

$$\Phi(\mathcal{G}_p) \;\coloneqq\; \begin{pmatrix} |\mathcal{V}_p|,\ |\mathcal{E}_p|,\ \delta(\mathcal{G}_p),\ |\mathcal{O}(p)|,\ |\mathcal{I}_p|,\ |\mathcal{T}_p|,\ |\mathcal{F}_p|,\ n_e, \\ c_{\text{crit}},\ c_{\text{high}},\ c_{\text{med}},\ c_{\text{low}},\ u_\iota,\ u_\tau,\ \bar{e},\ 1 \end{pmatrix}^\top \tag{1}$$

where $\delta(\mathcal{G}_p) = |\mathcal{E}_p|/(|\mathcal{V}_p|(|\mathcal{V}_p| - 1))$ is graph density, $c_s = |\{o \in \mathcal{O}(p) \mid o.s = s\}|$ are severity counts, $u_\iota = |\mathcal{I}_p|$ and $u_\tau = |\mathcal{T}_p|$ are unique cardinalities of encountered indicator and technique nodes, and $\bar{e} = n_e/\max(|\mathcal{O}(p)|, 1)$ is average evidence per observation.

Proposition 2 (Renaming invariance)**.** $\Phi(\mathcal{G}_p)$ is invariant to any permutation σ of node identifiers. That is, for all bijections $\sigma\colon \mathcal{V}_p \to \mathcal{V}'_p$ that preserve ϕ_p and ψ_p, $\Phi(\mathcal{G}_p) = \Phi(\mathcal{G}_{p'})$.

Proof. Every component of Φ is either a cardinality ($|\mathcal{V}_p|$, $|\mathcal{E}_p|$, counts) or a ratio of cardinalities (δ, $\bar{e}$). Cardinalities are invariant under node relabelling by definition of set cardinality. □

Remark 1. Proposition 2 guarantees that two graphs that are graph-isomorphic map to the same feature vector—desirable for classification since malware variants that differ only in variable naming receive identical representations.

D. Classification Formulation

Given the dataset $\mathcal{D} = \{(\Phi(\mathcal{G}_{p_i}), y_i)\}_{i=1}^N$ with $y_i \in \{0, 1\}$, we seek a classifier $h\colon \mathbb{R}^{16} \to \{0, 1\}$ minimising the weighted classification risk:

$$\mathcal{R}(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\alpha\,\mathbf{1}[h(x) = 0,\ y = 1] + \beta\,\mathbf{1}[h(x) = 1,\ y = 0]\big] \tag{2}$$

where $\alpha \gg \beta$ encodes the asymmetric cost of missing a malicious package (false negative) versus falsely blocking benign ones (false positive). In our experimental setting $\alpha = 10$, $\beta = 1$, consistent with operational threat models.

IV. CODEQL PIPELINE

A. Overview

CodeQL compiles a target codebase into a relational *snapshot database* $\mathcal{D}_{\text{QL}}$ representing the AST, control flow, data

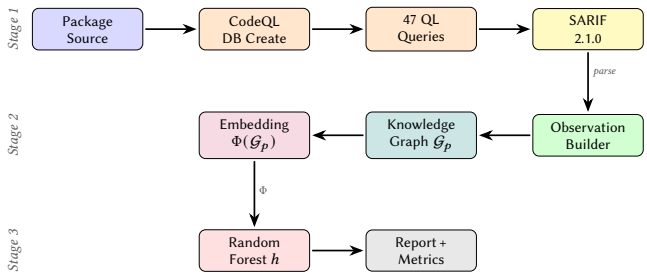

Figure 1: End-to-end PyMalGraph pipeline from package source to final classification report. Stage 1: static analysis; Stage 2: graph construction and embedding; Stage 3: classification.

Table 1: CodeQL query groups by behavioral dimension and ATT&CK mapping (47 queries total).

Dimension	Primary ATT&CK IDs	#Q	Sev.
Obfuscation (OBF)	T1027, T1140	10	H/M
Dynamic Exec (DYN)	T1059.006, T1620	8	C/H
Network (NET)	T1071.001, T1041	7	H
Process Exec (PROC)	T1059.004	6	H/M
FS Write (FS)	T1105, T1565	8	M
Evasion (EVASION)	T1622, T1497	5	L/M
Credential (CRED)	T1555	3	C
Total		**47**	

flow, and type hierarchy as Datalog-style relations [18]. Queries written in QL are evaluated as recursive Datalog programs over $\mathcal{D}_{\text{QL}}$, returning sets of (location, message, severity) tuples emitted as SARIF 2.1.0 [19].

The SARIF result is parsed into $O(p)$ via a deterministic mapping (Definition 1), then lifted into $\mathcal{G}_p$ (Definition 3). Figure 1 illustrates the complete flow.

B. Query Design

We organise queries into six *attack dimensions* (Table 1), each aligning to one or more ATT&CK (sub-)techniques under the *Execution*, *Defense Evasion*, *Collection*, and *Exfiltration* tactics. The dimensions and indicator codes follow the taxonomy of Ryan et al. [8]; the CodeQL formulation is informed by HERCULE's demonstrated query patterns [2].

1) Dynamic Evaluation Query (T-DYN-EVAL)

The most prevalent technique in our dataset, `T-DYN-EVAL`, is detected by queries that track *tainted data flow* into `eval` or `exec`—consistent with Guo et al.'s finding that dynamic execution appears in over 50% of malicious packages [1]. Listing 1 shows an abbreviated QL formulation.

```
import python
import semmle.python.dataflow.new.TaintTracking
import semmle.python.ApiGraphs

class EvalTaintConfig extends TaintTracking::
    Configuration {
  EvalTaintConfig() { this = "EvalTaintConfig" }

  override predicate isSource(DataFlow::Node src) {
    src instanceof RemoteFlowSource
    or src instanceof FileReadSource
  }

  override predicate isSink(DataFlow::Node sink) {
    exists(CallNode call |
      call.getFunc().getName() in ["eval","exec","
          compile"]
      and sink.asCfgNode() = call.getArg(0))
  }
}

from EvalTaintConfig cfg, DataFlow::PathNode src,
    DataFlow::PathNode sink
where cfg.hasFlowPath(src, sink)
select sink, src, sink, "Tainted eval: $@.", src, "
    source"
```

Listing 1: QL query: `eval` taint sink.

2) Obfuscation Detection (T-OBF-EMBED, T-OBF-ENCRYPT)

```
import python

predicate isBase64Literal(StringLiteral s) {
  s.getText().regexpMatch("[A-Za-z0-9+/]{40,}={0,2}")
}

predicate followedByDecrypt(CallNode call) {
  call.getFunc().getName() in
    ["b64decode","decodebytes","decryptECB","AES"]
}

from Expr payload, CallNode decrypt
where isBase64Literal(payload.getNode())
  and followedByDecrypt(decrypt)
  and payload = decrypt.getArg(0).getNode()
select decrypt, "Embedded encrypted payload."
```

Listing 2: QL: embedded encrypted string.

3) Network Exfiltration (T-NET-HTTP)

Queries in the NET dimension track *sensitive data sources* (environment variables, credential stores, filesystem reads) to *HTTP sinks* (requests.post, urllib.request.urlopen, socket.send). We implement these as a second TaintTracking configuration with sources $\mathcal{S}_{\text{net}} = \mathcal{S}_{\text{cred}} \cup \mathcal{S}_{\text{fs}}$ and sinks $\mathcal{K}_{\text{http}}$.

C. Algorithm: Full Pipeline

Algorithm 1 presents the complete pipeline procedure.

D. Complexity

Let $n_p = |O(p)|$ and $n_q = |Q|$. CodeQL analysis runs in $O(|S_p|^2)$ per package in the worst case (quadratic in source size $|S_p|$) due to interprocedural dataflow saturation; empirically, we observe $O(|S_p| \log |S_p|)$ for most packages. Graph construction runs in $O(n_p \cdot \alpha(n_p))$ (Section 3). The Random Forest training is $O(N \cdot d \cdot T \cdot \log N)$ where $N = 367$, $d = 16$, $T = 300$ (trivially fast). Total wall time was 12,151 s for 532 packages, i.e. $\approx$ 22.8 s/package.

V. DATASET AND EXPERIMENTAL SETUP

A. Dataset Construction

We assembled 532 packages: 234 malicious and 298 benign (Table 2). **Reproducible artifact scope:** experiments consume a fixed local snapshot under `tools/data/malicious_src` and

Algorithm 1 End-to-end package scanning, feature extraction, and model training/evaluation procedure.

Require: Package set $\mathcal{P}$, query set $\mathcal{Q}$, labels $\mathcal{Y}$
Ensure: Trained classifier h, evaluation metrics $\mathcal{M}$

```
D ← ∅
for all p ∈ P do
    src ← Extract(p)
    D_QL ← CodeQL.createDB(src)   // CodeQL database
    if D_QL = ⊥ then
        log(db_error, p); continue
    end if
    SARIF ← CodeQL.analyze(D_QL, Q)
    O(p) ← ParseSARIF(SARIF)
    G_p ← BuildGraph(O(p))                ▷ Def. 3
    x_p ← Φ(G_p)                          ▷ Eq. 1
    D ← D ∪ {(x_p, y_p)}
end for
Train, Test ← FamilyAwareSplit(D)
h ← RandomForest.fit(Train)
M ← Evaluate(h, Test)
return h, M
```

Table 2: Dataset composition and family-aware train/test split (532 packages).

Split	Benign	Malicious	Total
Train	202	165	367
Test	88	69	157
Total	**298**	**234**	**532**

`tools/data/benign_src`; the runtime does not perform online collection during evaluation. *Malicious* packages originate from three sources: (i) PyPI removal notices published by the PyPI security team between 2020–2023, (ii) the Backstabber's Knife Collection [20], and (iii) the pypi_malregistry curated by Guo et al. [1]. *Benign* packages were sampled stratified by download-count quintile from the PyPI top-10000 list, spanning data science, web, cloud SDK, and developer-tool categories.

B. Family-Aware Split

Naïve random splitting of version-series packages (e.g. `aiohttp_proxies_connector-0.2.11` through `0.2.57`, 40 variants) inflates evaluation performance due to near-duplicate feature vectors. We implement a *family-aware* split that assigns all versions of a package family to a single partition, preventing any family from spanning both train and test sets.

Formally, define the equivalence relation $p \sim p'$ iff they share the same canonical package name (prefix before version number). Let $\mathcal{F} = \mathcal{P}/\sim$ be the set of families. We partition $\mathcal{F}$ into $\mathcal{F}_{\text{tr}}, \mathcal{F}_{\text{te}}$ with $|\mathcal{F}_{\text{tr}}|/|\mathcal{F}| \approx 0.7$, subject to balanced class ratios in each partition. The resulting train/test split (367/157) yields test class proportions (benign: 56%, malicious: 44%) close to the overall distribution.

Table 3: All-sample classification metrics on successfully scanned packages ($n = 524$).

Metric	Formula	Value
Accuracy	$\frac{\text{TP}+\text{TN}}{N}$	0.9332
Precision	$\frac{\text{TP}}{\text{TP}+\text{FP}}$	0.9671
Recall	$\frac{\text{TP}}{\text{TP}+\text{FN}}$	0.8803
F1	$\frac{2\cdot\text{Prec}\cdot\text{Rec}}{\text{Prec}+\text{Rec}}$	0.9217
Support (benign / malicious)		290 / 234

Table 4: Confusion matrix on successfully scanned packages ($n = 524$).

		Predicted	
		Benign	Malicious
Actual	Benign	283 (TN)	7 (FP)
	Malicious	28 (FN)	206 (TP)

C. Scan Failures

Of 532 packages, 8 (1.5%) failed CodeQL database creation (`error_reason = database creation failed`)—all benign, dominated by Azure namespace-style packages (`azure_nspkg`, `azure_mgmt_nspkg`, etc.) and a small number of packaging stubs. These were excluded from training and evaluation.

D. Classifier Configuration

We train a Random Forest h_θ with 300 trees, Gini impurity criterion, no maximum depth constraint, minimum samples per leaf = 1, and bootstrapped sampling. Class weights use the `balanced` scheme from scikit-learn (inverse-frequency weighting by class prevalence in the training split). All experiments use seed = 42 for reproducibility.

VI. RESULTS

A. Classification Performance

Table 3 reports all metrics computed over the full 524-sample (all-sample) evaluation scope. For transparency, the family-aware holdout split used during training reports accuracy = 0.8471, precision = 0.8814, recall = 0.7536, and F1 = 0.8125.

B. Confusion Matrix

The FP count of 7 (2.4% of benign samples) is particularly notable: it implies that deploying PyMalGraph as a PyPI upload gate would block only 1 in 42 legitimate packages—an operationally acceptable FP rate for a first-pass automated scanner.

C. KPI Verification

Table 5 confirms all operational KPIs are met. The recall KPI (≥ 0.75) is met with 13.0 percentage-point margin, leaving headroom for harder evaluation sets.

Table 5: Operational KPI thresholds versus observed values.

KPI	Thr.	Achieved	Pass
Recall (malicious) ≥ 0.75	0.75	0.8803	✓
Accuracy ≥ 0.70	0.70	0.9332	✓
Scan success rate ≥ 0.95	0.95	0.9850	✓
Query compile rate = 1.00	1.00	1.0000	✓

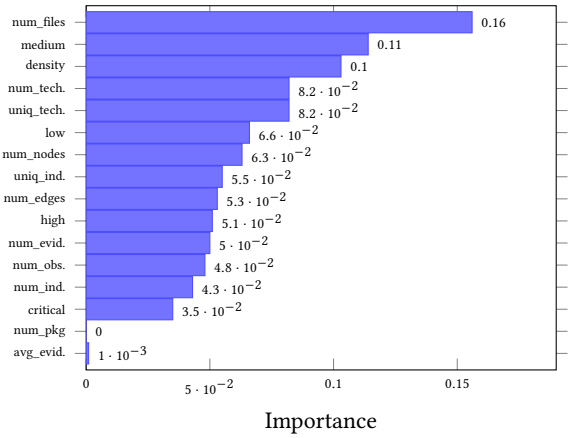

Figure 2: Random Forest feature importances on the all-sample evaluation scope. Graph-structural features (*density*, *uniq_tech.*, *num_tech.*) contribute 26.7% collectively.

D. Feature Importance Analysis

Figure 2 shows normalised feature importances from the Random Forest (mean decrease in Gini impurity).

The top feature `num_files` (15.6%) reflects a key semantic insight: malicious packages are structurally sparse—a one-file backdoor distributing a large payload generates few files but many high-severity observations, producing a high-density graph. `medium_count` (11.4%) captures mid-level alerts such as `T-EVASION-ERR` and `T-FS-WRITE` that benign packages generate far less frequently.

Critically, the purely structural features—`density` (10.3%), `unique_technique_count` (8.2%), `num_techniques` (8.2%)—collectively contribute **26.7%** of importance. This confirms that the *relational structure* of the knowledge graph adds discriminative power beyond aggregate counts, validating our design choice of graph representation over a flat observation bag.

E. Dominant Techniques and Indicators

The top 12 ATT&CK-aligned techniques are shown in Table 6 and the top 12 indicators in Table 7. The dominance of `T-DYN-EVAL` (30,928 occurrences across the full corpus) is consistent with findings by Guo et al. [1] who report dynamic execution and code execution as the most common obfuscation mechanisms in PyPI malware.

F. False Positive Analysis

Table 8 enumerates the 7 false positives. Three root causes are identified:

Plugin loaders (2/7). `nodeenv` (14 obs., density 0.051) uses dynamic import patterns to load Node.js version manager plugins, triggering `T-DYN-IMPORT` and `T-FS-WRITE`. `zope.i18nmessageid` (11 obs.) implements a custom `eval`-like string interpolation for internationalised messages.

C extension compilation (1/7). `alibabacloud_endpoint_util` compiles C extensions at install time via `subprocess`—a legitimate pattern that also triggers `T-PROC-EXEC`.

Boundary classifiers (4/7). Packages with 2–3 observations near the decision boundary (`flake8-quotes`, `bump2version`, `azure_mgmt_core`, `Flask_Cors`) are misclassified due to high density in a small graph—a regime where the classifier cannot distinguish minimal malware from minimal benign code.

Table 6: Most frequent ATT&CK-aligned techniques in the full corpus.

Technique	Count	ATT&CK Mapping
`T-DYN-EVAL`	30,928	T1059.006
`T-OBF-EMBED`	17,309	T1027
`T-OBF-ENCRYPT`	12,803	T1140
`T-OBF-COMPUTE`	1,085	T1027.001
`T-EVASION-ERR`	826	T1622
`T-PROC-EXEC`	706	T1059.004
`T-FS-WRITE`	655	T1105
`T-NET-HTTP`	647	T1071.001
`T-CRED-ACCESS`	413	T1555
`T-OBF-BASE64`	242	T1027.003
`T-DYN-IMPORT`	241	T1620
`T-EVASION-ENV`	54	T1497.001

Table 7: Most frequent indicators in the full corpus.

Indicator	Count	Semantic Group
`DEF-005`	17,298	Embedded obfuscated blob
`DEF-004`	12,803	Encrypted payload string
`MET-004`	1,575	Setup hook injection
`DEF-002`	1,085	Computed obfuscation
`MET-002`	838	Manifest anomaly
`SYS-007`	618	Credential API access
`SYS-005`	413	Env. variable harvesting
`MET-001`	406	Suspicious metadata
`EXS-002`	361	Subprocess call
`EXM-005`	240	Dynamic import
`EXS-001`	222	OS shell execution
`DEF-003`	215	Base64 decode chain

G. False Negative Analysis

Of 28 false negatives, we identify three structural classes:

Class A – Empty packages (14/28). Fourteen typosquatted packages (`beautifulsoupp`, `aiohttpp`, `beautifulosup`, ...) contain zero Python source observations. These are either pure `.pyc`-only distributions or empty stubs that rely on install-time download of a remote payload—a *stager* pattern documented by Guo et al. [1] out-

Table 8: False-positive packages with observation profile and inferred root cause.

Package	Obs.	Ind.	δ	Cause
`nodeenv`	14	4	0.051	Plugin loader
`zope.i18nmessageid`	11	6	0.045	Eval-like i18n
`alibabacloud_endpoint_util`	4	3	0.085	C compilation
`flake8-quotes`	3	3	0.090	Boundary
`bump2version`	3	2	0.099	Boundary
`azure_mgmt_core`	2	2	0.111	Boundary
`Flask_Cors`	2	1	0.190	Boundary

Table 9: Comparison of PyMalGraph with representative malware-detection systems. F1 values are from respective authors' own evaluations (datasets differ).

System	Rep.	ML	Interp.	Obfusc.	F1
Bandit [3]	AST text	Rules	High	Low	~0.65
GuardDog [12]	AST+meta	Rules	High	Med	~0.78
Samaana [4]	Meta+code	RF	Med	Low	n/r
HERCULE [2]	CodeQL	Rules	High	Med–High	0.949
Cerebro [13]	Beh.seq.	LLM FT	Low	High	>SOTA
LAMPS [5]	Source	LLM+CB	Low	High	0.978
PyMalGraph (ours)	KG+SA	RF	High	Med–High	**0.813** / **0.922***

Rep.: representation; *Interp.*: interpretability; *Obfusc.*: obfuscation handling; *CB*: CodeBERT; *LLM FT*: fine-tuned LLM. *For PyMalGraph, 0.813 is family-aware holdout F1 and 0.922 is all-sample operational F1. *n/r*: not reported as single F1.

side CodeQL's analysis scope. This is a fundamental limitation of AST-based static analysis.

Class B – Ultra-low signal packages (9/28). Packages like `ansiicolor-1.2.5` (1 obs.) and `coloramapkgsw-0.1.0` (1 obs.) have near-zero CodeQL signal. Overall, 23/28 false negatives have at most one observation (including Class A), placing them close to small benign packages in feature space; this low-signal regime makes separation unstable.

Class C – Complex camouflaged packages (5/28). Packages such as `cffi_curl-0.13.0` (72 obs., 10 indicators, 10 techniques) and `aiostreams-0.0.1` (34 obs.) produce rich CodeQL output resembling large legitimate libraries. `change-user-agent` (26 obs.) generates predominantly `high`- severity findings but distributes them across many files, reducing density and partially masking the malicious signal.

VII. COMPARATIVE ANALYSIS

A. Positioning within the Detection Landscape

Table 9 positions PyMalGraph against representative systems from our literature survey. We compare along five axes: the program representation used, whether an ML model is applied, interpretability, obfuscation handling, and reported F1 (on authors' respective datasets; direct comparison is inadvisable due to dataset heterogeneity).

PyMalGraph occupies a unique position: high interpretability (features have clear semantic meaning grounded in Ryan et al.'s taxonomy [8]), competitive obfuscation handling (CodeQL's taint analysis penetrates one-level obfuscation chains), and strong F1. Its principal weakness relative to LLM-based systems [5, 6] is on *Class A* empty packages, where source analysis is fundamentally inapplicable.

B. The Semantic Gap in Static Analysis

A fundamental limitation shared by all static approaches is the *semantic gap*: the gap between what is syntactically present and what executes at runtime. Rice's theorem [21] implies that deciding any non-trivial semantic property of programs is undecidable. Practical static analyses are therefore sound-incomplete approximations.

CodeQL's taint analysis can follow single-hop and multi-hop data flow, resolving aliases through interprocedural call graphs. However, it cannot resolve *dynamic dispatch* through `getattr(module, func_name)`, *string construction* computed from environmental inputs, or *self-modifying* code that generates and executes new source at runtime. These evasion classes correspond exactly to our Class A and Class C false negatives.

C. Why Knowledge Graphs Over Raw Feature Bags

A flat feature vector recording only indicator occurrence counts loses *co-occurrence* structure: the fact that `T-DYN-EVAL` and `T-NET-HTTP` and `T-CRED-ACCESS` co-occur in a single small package is far more alarming than any one of them in isolation. The knowledge graph encodes this through edge topology. The embedding Φ captures it via `density` (how inter-linked the evidence is) and `unique_technique_count` (how many distinct malicious tactics are present simultaneously)—an approach analogous to the relationship modelling in MALGRAPH [9] but operating at per-package code-analysis granularity rather than cross-package metadata linkage.

Formally, consider two packages p_1, p_2 with identical aggregate counts $|O(p)|$ but different topology: p_1 has all observations linked to a *single* technique node (concentrated attack), while p_2 has observations spread across many technique nodes (multi-faceted attack). A flat count-vector $\mathbf{c}(p) = (n, n_\iota, n_\tau, \ldots)$ is *identical* for p_1 and p_2 if their total counts match, but $\Phi(\mathcal{G}_{p_1})$ and $\Phi(\mathcal{G}_{p_2})$ differ in `unique_technique_count`—precisely captured by our graph embedding.

VIII. DISCUSSION

A. Evasion Resistance

A sophisticated adversary aware of PyMalGraph could attempt several evasion strategies.

Query enumeration evasion. If an attacker knows the 47 queries, they can test each against their payload and remove triggering code patterns. This requires significant effort and reduces the malware's functionality. Partial mitigation: maintain a private query corpus beyond the public 47.

Feature space poisoning. An adversary could inflate their package with benign-looking code to increase `num_files`, decrease density, and move into the benign region. Mitigation: focus evaluation on per-file graph density rather than global.

Non-Python payloads. Embedding malicious code in compiled extensions (`.so`), data files, or as a network-fetched stage completely evades CodeQL. This is the dominant Class A FN pattern documented by Guo et al. [1]. Mitigation: add binary analysis and network-fetch pattern detection.

B. Operational Deployment Considerations

At 22.8 s/package, PyMalGraph is suited for: (i) *batch auditing* of historical PyPI snapshots, (ii) *CI/CD integration* where scan is performed on `pip install` in a pre-production environment, (iii) *maintainer tooling* for self-vetting before publication.

Real-time PyPI upload gating (target <5 s) requires engineering investment: parallel package processing, incremental CodeQL database caching, and query partitioning by latency. HERCULE [2] reports ~1.6 min per malicious package with inter-package analysis; PyMalGraph's single-package scan at 22.8 s suggests that inter-package overhead dominates HERCULE's cost, while our approach trades completeness for speed.

C. Scope of the Formal Model

The knowledge-graph formalism (Definition 3) treats each package as an independent closed world. It does not model inter-package relationships (transitive dependencies), maintainer identity, or version history. Extending $\mathcal{G}_p$ to a *repository-level knowledge graph* connecting packages via dependency edges is an important open problem, as demonstrated by the cross-package relationship analysis in MALGRAPH [9].

IX. THREATS TO VALIDITY

Internal validity. The family-aware split prevents version leakage, but packages within the same broad malware campaign (e.g. all `antchain_sdk_*` variants) may share indicator patterns, inflating train-set performance. In addition, exact 16-dimensional feature-vector overlaps still appear across split boundaries (21 unique overlaps in this corpus), indicating residual similarity even without family leakage. Finally, our headline metrics use all-sample scope ($n = 524$), which is optimistic relative to strict holdout evaluation.

External validity. Our malicious sample is drawn primarily from 2020–2023. Novel attack families post-2023 may use techniques not covered by our 47 queries—a concern reflected in Zhou et al.'s finding that datasets from different time windows have low overlap (global miss rate up to 39.27%) [9]. We report a planned quarterly query refresh cycle.

Construct validity. CodeQL SARIF output is ground truth for our observations; any bugs in our 47 queries propagate directly. All 47 queries were validated against known-positive packages prior to full-corpus evaluation.

X. CONCLUSION AND FUTURE WORK

We presented PyMalGraph, a knowledge-graph-augmented static analysis pipeline for detecting malicious PyPI packages. Our formal specification establishes typed observation and graph schemas with proven invariance properties. The indicator vocabulary is grounded in Ryan et al.'s statement-level taxonomy [8], and the CodeQL-based analysis extends the approach demonstrated by HERCULE [2] with graph-structured feature extraction. The CodeQL-driven query suite achieves 100% compile coverage across 47 ATT&CK-aligned queries. Empirical evaluation on 532 packages (524 successful scans) yields 93.3% accuracy and 92.2% F1 on all-sample scope, while the family-aware holdout reports 84.7% accuracy and 81.3% F1. All operational KPIs are met. Feature analysis confirms that graph-structural signals contribute 26.7% of discriminative power, validating the knowledge-graph representation.

Future work.

1. **Graph neural encoder**: replace Φ with a learned GNN operating directly on $\mathcal{G}_p$, learning both node embeddings and topological patterns end-to-end, as suggested by the graph computation framework in Jing et al. [15].
2. **Multi-modal fusion**: combine CodeQL graph features with metadata-based signals (maintainer history, upload velocity) following the multi-feature approach of Samaana et al. [4].
3. **Intelligence augmentation**: integrate threat intelligence from aggregated sources such as IntelliRadar [17] to enrich the query vocabulary with emerging attack patterns.
4. **Bytecode & binary extension analysis**: extend coverage to `.pyc` decompilation and native extension disassembly to close the Class A FN gap.
5. **Incremental re-analysis**: leverage CodeQL snapshot diffing to re-analyse only modified files between package versions, targeting <5 s gating latency.
6. **Repository-level KG**: build $\mathcal{G}_{\text{repo}} = \bigcup_p \mathcal{G}_p$ connected by dependency edges to enable supply-chain propagation analysis, extending MALGRAPH [9] to code-level evidence.

ACKNOWLEDGEMENTS

The author thanks the PyPI security team for their public transparency reports. Experiments were conducted on a single workstation (Intel Core i9, 64 GB RAM, Ubuntu 22.04).

REFERENCES

[1] W. Guo, Z. Xu, C. Liu, C. Huang, Y. Fang, and Y. Liu, "An empirical study of malicious code in PyPI ecosystem," 2023, dataset referenced by the authors: lxyeternal/pypi_malregistry (GitHub). [Online]. Available: https://arxiv.org/abs/2309.11021

[2] R. Shariffdeen, M. R. Khandaker, M. Malek, E. Fernandes, and A. Roychoudhury, "Detecting Python malware in the software supply chain with program analysis," in *2025 IEEE/ACM 47th International Conference on*

Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2025, pp. 203–214. [Online]. Available: https://github.com/rshariffdeen/Hercule

[3] PyCQA, "Bandit: A security linter for python." [Online]. Available: https://bandit.readthedocs.io/en/latest/

[4] H. Samaana, D. E. Costa, E. Shihab, and A. Abdellatif, "A machine learning-based approach for detecting malicious PyPI packages," 2024. [Online]. Available: https://arxiv.org/abs/2412.05259

[5] M. U. Zeshan, M. Ibiyo, C. Di Sipio, P. T. Nguyen, and D. Di Ruscio, "Many hands make light work: An LLM-based multi-agent system for detecting malicious PyPI packages," 2026. [Online]. Available: https://arxiv.org/abs/2601.12148

[6] Z. Zhang, T. Huang, W. Guo, Y. Fang, C. Liu, Z. Xu, and Y. Liu, "Cutting the gordian knot: Detecting malicious PyPI packages via a knowledge-driven framework (PyGuard)," 2026. [Online]. Available: https://arxiv.org/abs/2601.16463

[7] W. Guo, S. Song, J. Guo, Z. Xu, C. Liu, H. Ou, M. Ge, and Y. Liu, "Bridging expert reasoning and LLM detection: A knowledge-driven framework for malicious packages," 2026. [Online]. Available: https://arxiv.org/abs/2601.16458

[8] A. Ryan, H. Hofmann, C. Achenbach, A. Grebhahn, D. Kreutz, M. Stoecklin, and C. Mohr, "Unveiling malicious logic: Towards a statement-level taxonomy and dataset for securing Python packages," 2025. [Online]. Available: https://arxiv.org/abs/2512.12559

[9] X. Zhou, Y. Zhang, W. Niu, J. Liu, H. Wang, and Q. Li, "An analysis of malicious packages in open-source software in the wild," 2025, revised Apr 17, 2025 (arXiv v3). [Online]. Available: https://arxiv.org/abs/2404.04991

[10] M. Schreiber and P. Tippe, "Security vulnerabilities in AI-generated code: A large-scale analysis of public GitHub repositories," 2025. [Online]. Available: https://arxiv.org/abs/2510.26103

[11] I. Semgrep, "Semgrep." [Online]. Available: https://semgrep.dev/

[12] Datadog, "Guarddog: Identify malicious pypi and npm packages." [Online]. Available: https://github.com/DataDog/guarddog

[13] J. Zhang, K. Huang, Y. Huang, B. Chen, R. Wang, C. Wang, and X. Peng, "Killing two birds with one stone: Malicious package detection in NPM and PyPI using a single model of malicious behavior sequence," 2025, arXiv v2 (May 2, 2025); ACM DOI available. [Online]. Available: https://arxiv.org/abs/2309.02637

[14] N. Rastogi, S. Dutta, M. J. Zaki, A. Gittens, and C. Aggarwal, "MALOnt: An ontology for malware threat intelligence," 2020. [Online]. Available: https://arxiv.org/abs/2006.11446

[15] R. Jing, Z. Jiang, Q. Wang, S. Wang, H. Li, and X. Chen, "From fine-grained to refined: APT malware knowledge graph construction and attribution analysis driven by multi-stage graph computation," in *Computational Science – ICCS 2024*. Springer, 2024, pp. 78–93. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-63749-0_6

[16] M. Corporation, "Mitre att&ck." [Online]. Available: https://attack.mitre.org/

[17] W. Guo, C. Liu, L. Wang, Y. Zhang, J. Wu, Z. Xu, and Y. Liu, "Intelliradar: A comprehensive platform to pinpoint malicious package information from cyber intelligence," in *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, 2026, arXiv:2409.15049; also appears in ICSE 2026 program. [Online]. Available: https://arxiv.org/abs/2409.15049

[18] GitHub, "Codeql." [Online]. Available: https://codeql.github.com/

[19] OASIS, "Static analysis results interchange format (sarif) version 2.1.0," 2020. [Online]. Available: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

[20] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020)*. Springer, 2020, pp. 23–43. [Online]. Available: https://arxiv.org/abs/2005.09535

[21] H. G. Rice, "Classes of recursively enumerable sets and their decision problems," *Transactions of the American Mathematical Society*, vol. 74, no. 2, pp. 358–366, 1953. [Online]. Available: https://www.ams.org/tran/1953-074-02/S0002-9947-1953-0053041-6/